

Field-Oriented Control of a Squirrel-Cage Induction Motor

Simulation study — modeling, vector control design and multi-quadrant validation

July 2026 — companion report to the simulations in `python/` and `matLab/`

Abstract

This report presents the design and simulation of a Field-Oriented Control (FOC) scheme for a 5 kW, 380 V squirrel-cage induction motor. The machine is modeled by its two-axis (dq) dynamic equations in the stationary frame; the controller performs indirect rotor-flux orientation with cascaded PI loops, space-vector modulation and full decoupling feed-forward. The drive is validated on a staircase of constant-speed setpoints — reference steps to 800, 1400 and 600 rpm followed by a reversal step to -800 rpm, with a 10 N·m load-torque disturbance — demonstrating independent flux/torque regulation and four-quadrant operation. Each speed step settles within 70–95 ms with no overshoot; between transients the speed error stays below 0.25 rpm (0.05 rpm RMS), the load step is rejected with a 69 rpm dip recovered in about 70 ms, and the rotor flux holds its 0.8 Wb reference with a q-axis residual under 2 %, confirming correct field orientation throughout. A second, control-oriented scenario (ramp and sinusoidal references, unloaded) isolates the servo behavior of the PI speed loop and verifies the internal model principle quantitatively: zero steady-state error on the ramp (type-2 loop) and a frequency-dependent sinusoidal tracking error matching the analytical prediction within ≈ 9 %. The drive is then rebuilt twice in Simulink — once from fundamental blocks and once from Simscape Electrical components with a true 10 kHz switching bridge — and the three independent implementations agree on every physical quantity.

1. Introduction

The three-phase induction machine is the workhorse of industry: rugged, brushless and inexpensive. Its drawback is control complexity — unlike a DC machine, where the field flux and the armature (torque) current are held by two separate, naturally orthogonal circuits, the induction machine develops torque through the interaction of quantities that are all AC, all coupled and all embedded in one set of nonlinear differential equations. Simply varying the supply voltage or frequency (scalar V/f control) leaves torque dynamics slow and poorly damped.

Field-Oriented Control (vector control), introduced by Blaschke and Hasse in the early 1970s, resolves this by controlling the machine *in a coordinate frame attached to the rotor flux*. In that frame, and only there, the machine behaves like a separately-excited DC machine: one current component (i_d) sets the flux, the other (i_q) sets the torque, and both are DC quantities in steady state — so ordinary PI controllers regulate them with zero error.

This study covers the complete chain: machine model, coordinate transformations (Clarke, Park), indirect FOC structure, controller tuning, and simulation results over a multi-level speed-step profile including a reversal. Constant setpoints reached through steps are deliberately chosen as the test class: they are the reference type a PI-based cascade regulates with exactly zero steady-state error, so every deviation seen in the results is attributable to physics (torque limits, load disturbances), not to a structural tracking limitation of the controller.

2. Induction machine model

2.1 Machine data

Quantity	Symbol	Value
Nominal power	P_n	5000 W
Supply voltage (line-line, rms)	U	380 V
Nominal current	I	10.5 A
Pole pairs	p	2
Stator resistance	R_s	0.435 Ω
Rotor resistance (referred)	R_r	0.816 Ω
Stator inductance (total)	L_s	0.071 H
Rotor inductance (total)	L_r	0.071 H
Mutual inductance	L_m	0.069 H
Inertia	J	0.0056 kg·m ²
Viscous friction	f_r	0.0015 N·m·s

Derived quantities used throughout (matching the intermediate values σ , τ_s , τ_r of the course notes):

$$\sigma = 1 - \frac{L_m^2}{L_s L_r} = 0.0555, \quad \tau_r = \frac{L_r}{R_r} = 87.0 \text{ ms}, \quad \tau_s = \frac{L_s}{R_s} = 163.2 \text{ ms}$$

$$\sigma L_s = 3.94 \text{ mH} \quad (\text{transient inductance seen by the current loops})$$

2.2 Dynamic dq model

Applying the transformations of Section 3 to the phase equations, the machine reduces to a fourth-order electrical system plus the mechanical equation. The simulation uses the **stationary ($\alpha\beta$) frame** with stator currents and rotor flux linkages as states — a numerically convenient, FOC-friendly choice:

$$\begin{aligned} \frac{d\psi_{r\alpha}}{dt} &= -\frac{\psi_{r\alpha}}{\tau_r} - \omega_r \psi_{r\beta} + \frac{L_m}{\tau_r} i_{s\alpha} \\ \frac{d\psi_{r\beta}}{dt} &= -\frac{\psi_{r\beta}}{\tau_r} + \omega_r \psi_{r\alpha} + \frac{L_m}{\tau_r} i_{s\beta} \\ \frac{di_{s\alpha}}{dt} &= \frac{1}{\sigma L_s} \left[v_{s\alpha} - R_\sigma i_{s\alpha} + \frac{L_m}{L_r \tau_r} \psi_{r\alpha} + \frac{L_m}{L_r} \omega_r \psi_{r\beta} \right] \\ \frac{di_{s\beta}}{dt} &= \frac{1}{\sigma L_s} \left[v_{s\beta} - R_\sigma i_{s\beta} + \frac{L_m}{L_r \tau_r} \psi_{r\beta} - \frac{L_m}{L_r} \omega_r \psi_{r\alpha} \right] \end{aligned}$$

with $R_\sigma = R_s + R_r (L_m/L_r)^2 = 1.206 \Omega$ and $\omega_r = p \omega_m$ the electrical rotor speed. Torque and mechanics (amplitude-invariant convention, hence the 3/2 factor):

$$T_e = \frac{3}{2} p \frac{L_m}{L_r} (\psi_{r\alpha} i_{s\beta} - \psi_{r\beta} i_{s\alpha}), \quad J \frac{d\omega_m}{dt} = T_e - T_L - f_r \omega_m$$

Relation to the course's matrix formulation. The state-space matrices A_1 , A_2 , A_3 , B , R , L given in the course notes describe the same machine with the state vector built from stator *and* rotor currents

$[i_{s\alpha}, i_{s\beta}, i_{r\alpha}, i_{r\beta}]^T$ (note the input matrix rows $1/\sigma L_s$ and $-L_m/\sigma L_s L_r$, and the diagonal terms $-1/\sigma\tau_s$, $-1/\sigma\tau_r$). The two formulations are strictly equivalent through the flux definition $\vec{\psi}_r = L_r \vec{i}_r + L_m \vec{i}_s$; the flux-based form is preferred here because the rotor flux is precisely the quantity FOC orients on, and because the flux states vary slowly (time constant τ_r), which conditions the integration well. The 3×3 matrix **T** of the notes is the three-phase projection that removes the homopolar component — it is exactly the Clarke transformation of the next section written for all three axes.

3. Reference-frame transformations

3.1 Why transform at all?

The stator carries three currents i_a, i_b, i_c displaced by 120° . For a machine with an isolated neutral, $i_a + i_b + i_c = 0$: only **two** of the three quantities are independent. Moreover, in any fixed frame all electrical quantities are sinusoidal at the supply frequency, which makes them awkward to regulate — a PI controller cannot track a sinusoid with zero error. The two transformations address these two problems *separately*, which is exactly why they are applied in sequence.

3.2 Clarke transformation ($abc \rightarrow \alpha\beta$): remove the redundancy

The Clarke transform projects the three 120° -spaced phase axes onto two orthogonal stationary axes (α aligned with phase a , β in quadrature), discarding the zero-sequence component. In the amplitude-invariant form used here:

$$\begin{bmatrix} x_\alpha \\ x_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} x_a \\ x_b \\ x_c \end{bmatrix}$$

The factor $2/3$ makes the $\alpha\beta$ amplitudes equal to the peak phase amplitude (the same convention as the course's matrix **T**). After Clarke, the machine is described by a *two-phase equivalent*, but the quantities are still AC: the current vector $\vec{i}_s = i_{s\alpha} + j i_{s\beta}$ rotates at the supply frequency.

3.3 Park transformation ($\alpha\beta \rightarrow dq$): stop the rotation

The Park transform re-expresses the $\alpha\beta$ vector in a frame rotating at the angle θ (for FOC, the rotor-flux angle θ_e):

$$\begin{bmatrix} x_d \\ x_q \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_\alpha \\ x_\beta \end{bmatrix}$$

Seen from a frame that rotates *with* the flux vector, steady-state sinusoids become **constants**. This is the decisive step: DC quantities can be regulated by PI controllers with zero steady-state error, limits and feed-forward terms become meaningful constants, and the flux/torque structure of the machine becomes visible (Section 4).

3.4 Why Clarke *then* Park?

Conceptually the pair is a single transformation $abc \rightarrow dq$, but performing it in two stages is both clearer and cheaper:

1. **Different jobs.** Clarke handles the *geometry* of the winding (3 axes $\rightarrow$ 2 orthogonal axes, zero-sequence removed); Park handles the *dynamics* (stationary $\rightarrow$ rotating). Keeping them separate isolates a constant, purely algebraic step from the angle-dependent step.

2. **Computation.** Clarke has constant coefficients — no trigonometry. Only Park needs one $\sin \theta_e / \cos \theta_e$ pair per sampling period, shared by the measurement path ($abc \rightarrow dq$) and the actuation path ($dq \rightarrow abc$). This is how every practical drive implements it.
3. **Intermediate signals are useful.** The $\alpha\beta$ quantities are exactly what the space-vector modulator needs (Section 4.4), and what stationary-frame observers use in sensorless drives. The chain $abc \rightarrow \alpha\beta \rightarrow dq$ exposes them for free.

The inverse path (inverse Park, then inverse Clarke) converts the controller's DC voltage commands v_d^*, v_q^* back into the three-phase quantities the inverter must synthesize.

4. Field-oriented control

4.1 Principle: rotor-flux orientation

Choose the d-axis of the Park frame *along the rotor flux vector*. Then by construction $\psi_{rq} = 0$, $\psi_{rd} = \psi_r$, and the machine equations collapse to:

$$\tau_r \frac{d\psi_r}{dt} + \psi_r = L_m i_d \quad (\text{flux is governed by } i_d \text{ alone, lag } \tau_r)$$

$$T_e = \frac{3}{2} p \frac{L_m}{L_r} \psi_r i_q \quad (\text{torque is proportional to } i_q \text{ at constant flux})$$

This is the DC-machine analogy: i_d plays the role of the field current, i_q of the armature current. Independent flux and torque control follows — *provided the frame really is aligned with the rotor flux*, i.e. provided θ_e is known accurately.

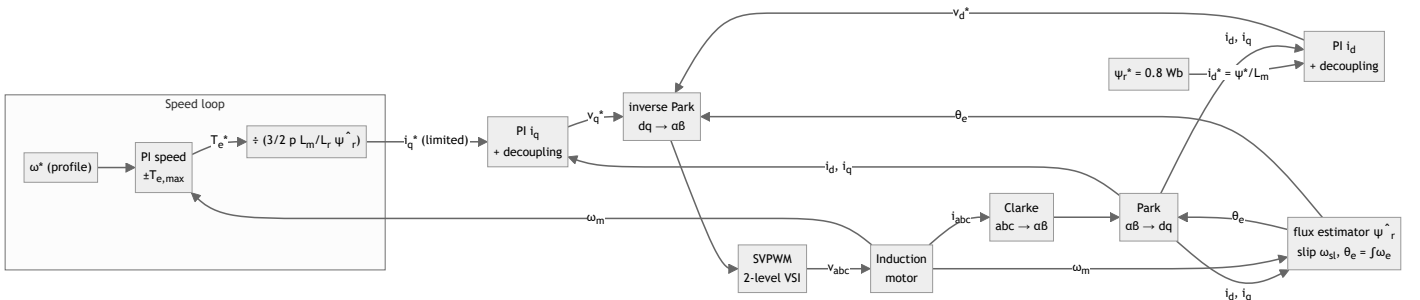
4.2 Indirect FOC: finding the flux angle without a flux sensor

Rotor flux is not measurable in a standard machine. The **indirect** method (Hasse) computes its angle from the slip relation that holds exactly when the frame is oriented:

$$\omega_{sl} = \frac{L_m i_q}{\tau_r \psi_r}, \quad \theta_e = \int (p \omega_m + \omega_{sl}) dt$$

The flux magnitude needed in these formulas is *estimated* through the rotor lag itself (current-model estimator): $\hat{\psi}_r = \frac{L_m}{1+s\tau_r} i_d$. Using the estimate (rather than the reference) keeps the torque scaling and the slip correct during flux build-up. Note the scheme's known sensitivity: both formulas contain τ_r , which varies with rotor temperature — the classical robustness limit of indirect FOC.

4.3 Control architecture



Reading the diagram left to right: the outer **speed PI** converts the speed error into a torque demand, which the estimated flux converts into i_q^* ; the flux channel needs no PI — in steady state $i_d^* = \psi_r^*/L_m$ feeds the rotor lag directly. The two **inner current PIs** (one per axis, sampled at 10 kHz) output voltage commands, augmented by **decoupling feed-forward** that cancels the rotational cross-coupling between the axes:

$$v_{d,ff} = -\omega_e \sigma L_s i_q, \quad v_{q,ff} = +\omega_e \left(\sigma L_s i_d + \frac{L_m}{L_r} \hat{\psi}_r \right)$$

so each PI sees an approximately first-order R–L plant. All PIs carry back-calculation **anti-windup**; the torque command is limited to ± 35 N·m, the current vector to 25 A (q-axis gets what the d-axis leaves: $i_{q,max} = \sqrt{I_{max}^2 - i_d^{*2}}$), and the voltage vector to the SVPWM linear limit with d-axis priority (flux integrity first).

4.4 Inverter and space-vector modulation

The two-level voltage-source inverter is fed from $V_{dc} = 540$ V ($\approx \sqrt{2} \cdot 380$, a diode rectifier bus). The modulator adds the min–max common-mode injection

$$v_{cm} = -\frac{1}{2} (\max(v_a^*, v_b^*, v_c^*) + \min(v_a^*, v_b^*, v_c^*))$$

which reproduces space-vector PWM duty cycles and extends the linear range to a phase amplitude of $V_{dc}/\sqrt{3} = 311.8$ V (15 % beyond sinusoidal PWM). The simulation uses the cycle-averaged inverter model (the switching model is available as a configuration option; it changes ripple, not the dynamics studied here).

4.5 Controller tuning

With decoupling active, each current loop sees $G_i(s) = 1/(\sigma L_s s + R_\sigma)$. Choosing the PI zero to cancel the plant pole and setting the open-loop crossover at ω_{ci} gives:

Loop	Design rule	Bandwidth	Gains
Current (d, q)	$K_p = \sigma L_s \omega_{ci}$, $K_i = R_\sigma \omega_{ci}$	400 Hz	$K_p = 9.9$ V/A, $K_i = 3030$ V/(A·s)
Speed	$K_p = J \omega_{cs}$, $K_i = K_p \omega_{cs}/5$	30 Hz	$K_p = 1.06$ N·m·s, $K_i = 39.8$ N·m

The $13\times$ separation between the loops keeps the cascade well-behaved; the sampling (10 kHz, equal to the PWM frequency) is $25\times$ the current-loop bandwidth. The speed-loop bandwidth was chosen against the load-rejection requirement: with this small inertia, a 10 N·m step at 15 Hz bandwidth would dip ≈ 180 rpm; at 30 Hz the simulated dip is 69 rpm.

5. Simulation setup

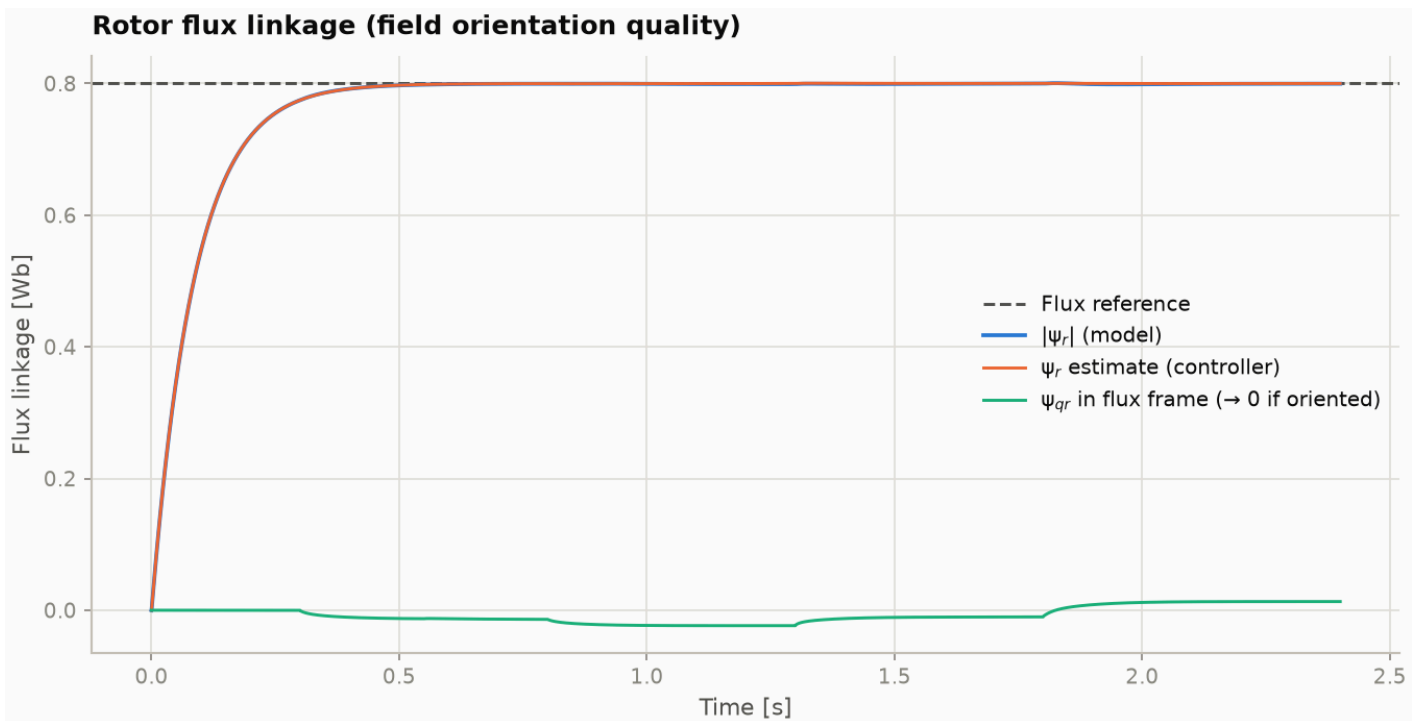
The plant integrates with a fixed-step RK4 solver (20 μ s, 5 sub-steps per control period); the controller runs strictly discretely at $T_s = 100$ μ s — no controller state is updated between samples, as in a real DSP implementation. Measurements (currents, speed) are ideal.

Test profile — a staircase of constant speed levels, every change applied as a sudden reference step (all levels editable in `python/main.py`):

Interval	Reference	Purpose
0 – 0.30 s	0 rpm (flux build-up)	magnetize: ψ_r reaches 0.8 Wb in $\approx 3\tau_r$
0.30 s	step 0 $\rightarrow$ 800 rpm	acceleration from standstill, torque saturation
0.55 s	—	load step 10 N·m , kept for the rest of the run
0.80 s	step 800 $\rightarrow$ 1400 rpm	acceleration under load
1.30 s	step 1400 $\rightarrow$ 600 rpm	deceleration (regenerative braking)
1.80 s	step 600 $\rightarrow$ –800 rpm	four-quadrant reversal through zero speed
... 2.40 s	hold –800 rpm	steady reversed operation under load

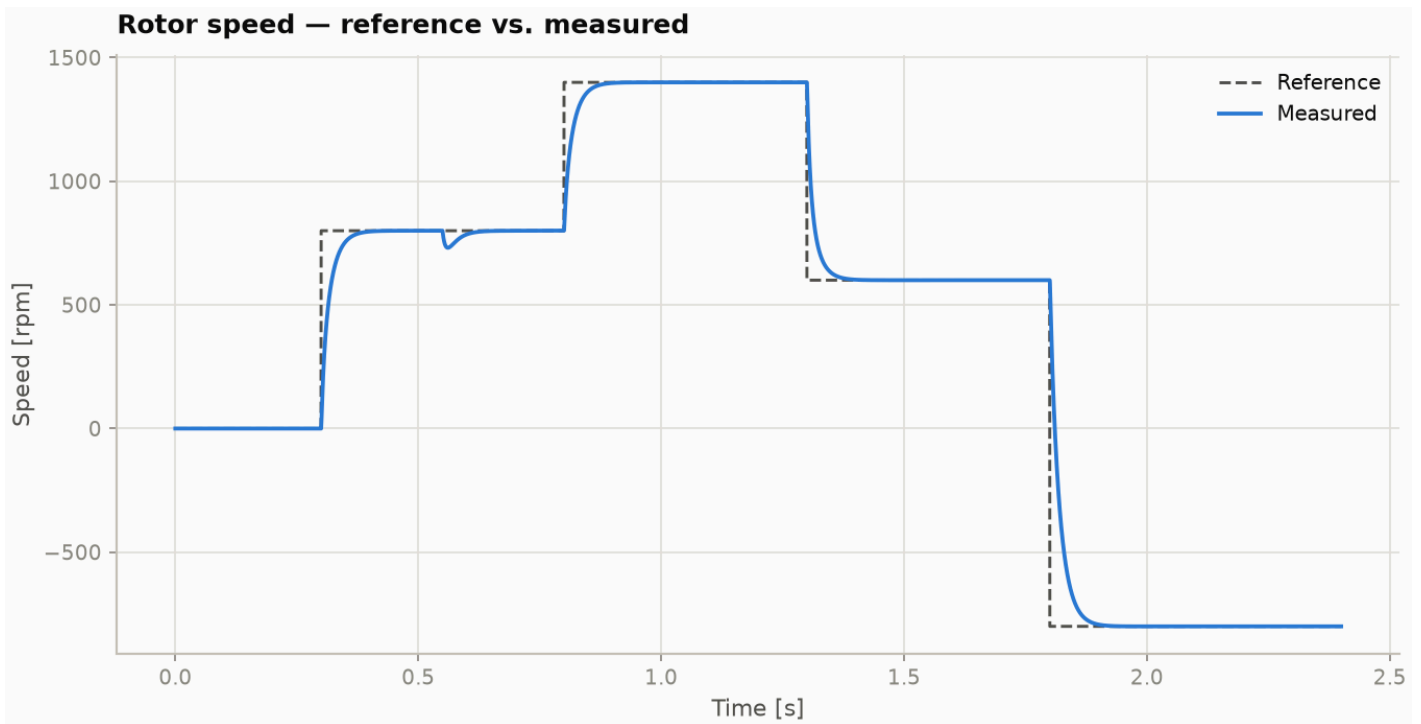
6. Results and discussion

6.1 Rotor flux and field orientation



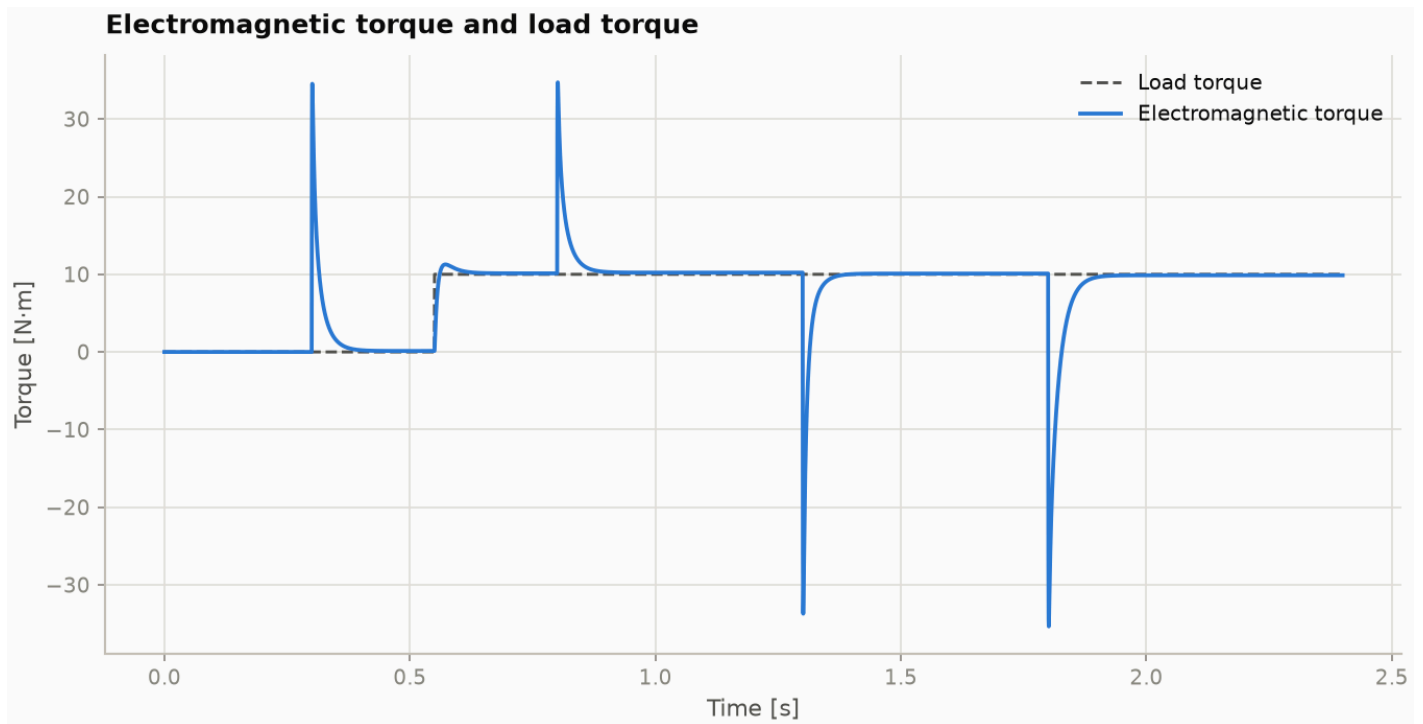
The flux rises with the expected first-order response ($\tau_r = 87$ ms, settled within ≈ 0.26 s) and stays pinned to 0.8 Wb for the remaining two seconds — through all four speed steps, the load engagement and the reversal. Two observations confirm correct orientation: the controller's estimate lies on top of the model's true flux magnitude, and the q-axis flux component seen from the controller's frame (green) never exceeds ≈ 0.02 Wb (2.5 % of rated), returning to ≈ 0.013 Wb at the end. A perfectly oriented drive would show exactly zero; the small residual is the transient orientation error of the indirect method during fast slip changes and is the expected signature of a healthy IFOC.

6.2 Speed tracking



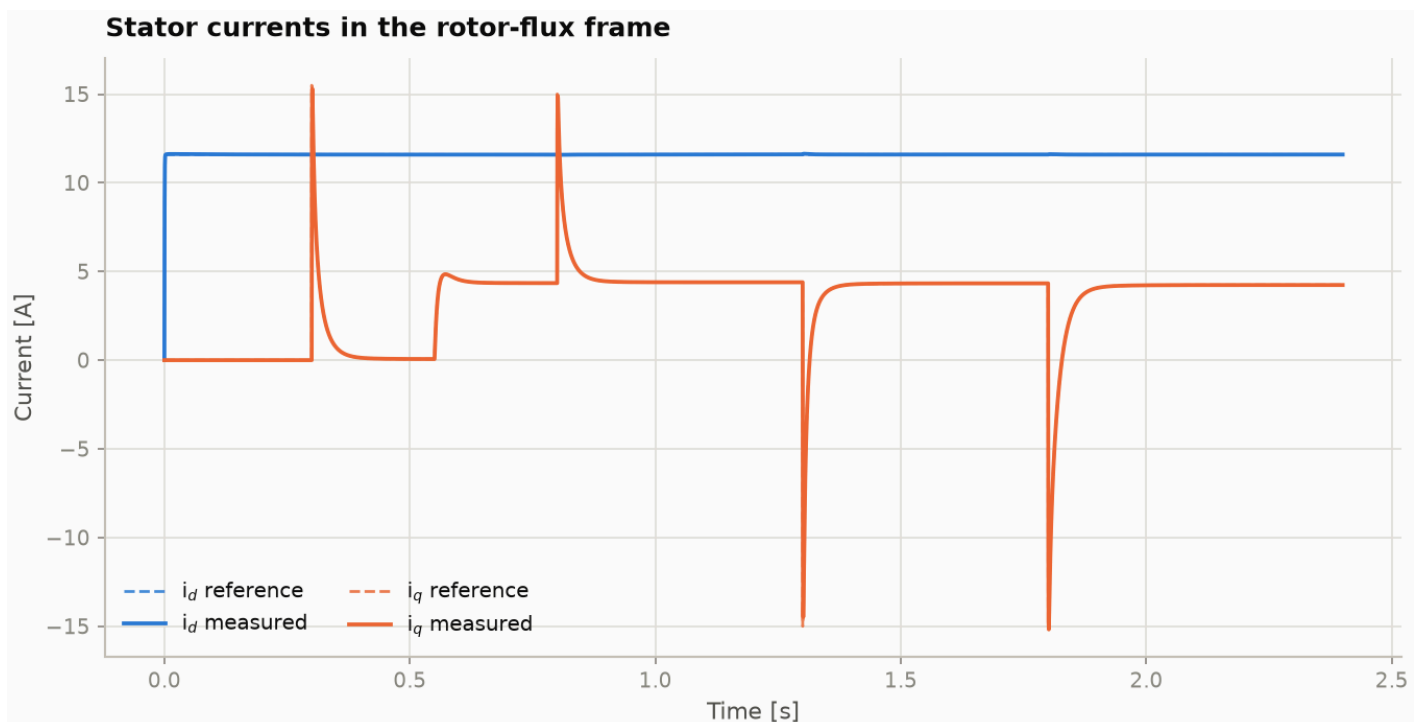
Each reference step produces the same clean signature: a torque-limited transient followed by an exponential approach to the new level with no overshoot and no steady-state error. Measured settling times (into a ± 10 rpm band): **79 ms** for $0 \rightarrow 800$ rpm, **75 ms** for $800 \rightarrow 1400$ rpm, **71 ms** for $1400 \rightarrow 600$ rpm and **95 ms** for the 1400-rpm-wide reversal step $600 \rightarrow -800$ rpm. During the transients the response speed is set by the 35 N·m torque ceiling, not by the loop gains — the speed rises along a straight (constant-acceleration) segment while the torque saturates, then the linear loop takes over. The reversal is handled with no special logic: the drive brakes, passes through zero speed and re-motors in the opposite direction — the four-quadrant capability vector control provides. Between transients the error essentially vanishes: **0.05 rpm RMS / 0.25 rpm maximum** over all settled portions. This is the internal-model principle at work: constant setpoints are exactly the reference class the PI integrator reproduces, hence zero steady-state error on every plateau. The 10 N·m load step at 0.55 s causes a **69 rpm dip recovered in ≈ 70 ms** — the disturbance-rejection benefit of the 30 Hz outer-loop bandwidth chosen in Section 4.5.

6.3 Torque



The torque trace reads like a commentary on the profile: a saturated +35 N·m spike drives each upward speed step, a step to 10 N·m marks the load engaging at 0.55 s, and the two downward transitions (1400 → 600 rpm and the reversal) produce saturated spikes at **-35 N·m** — the machine brakes regeneratively, working briefly as a generator while the kinetic energy of the shaft is returned through the inverter. After every spike the torque recovers cleanly with no hangover: the anti-windup keeps the PI integrators from charging during saturation. On each plateau the torque settles at the load line $T_L + f_r \omega_m$ (e.g. ≈ 9.9 N·m at -800 rpm, where the friction term is negative).

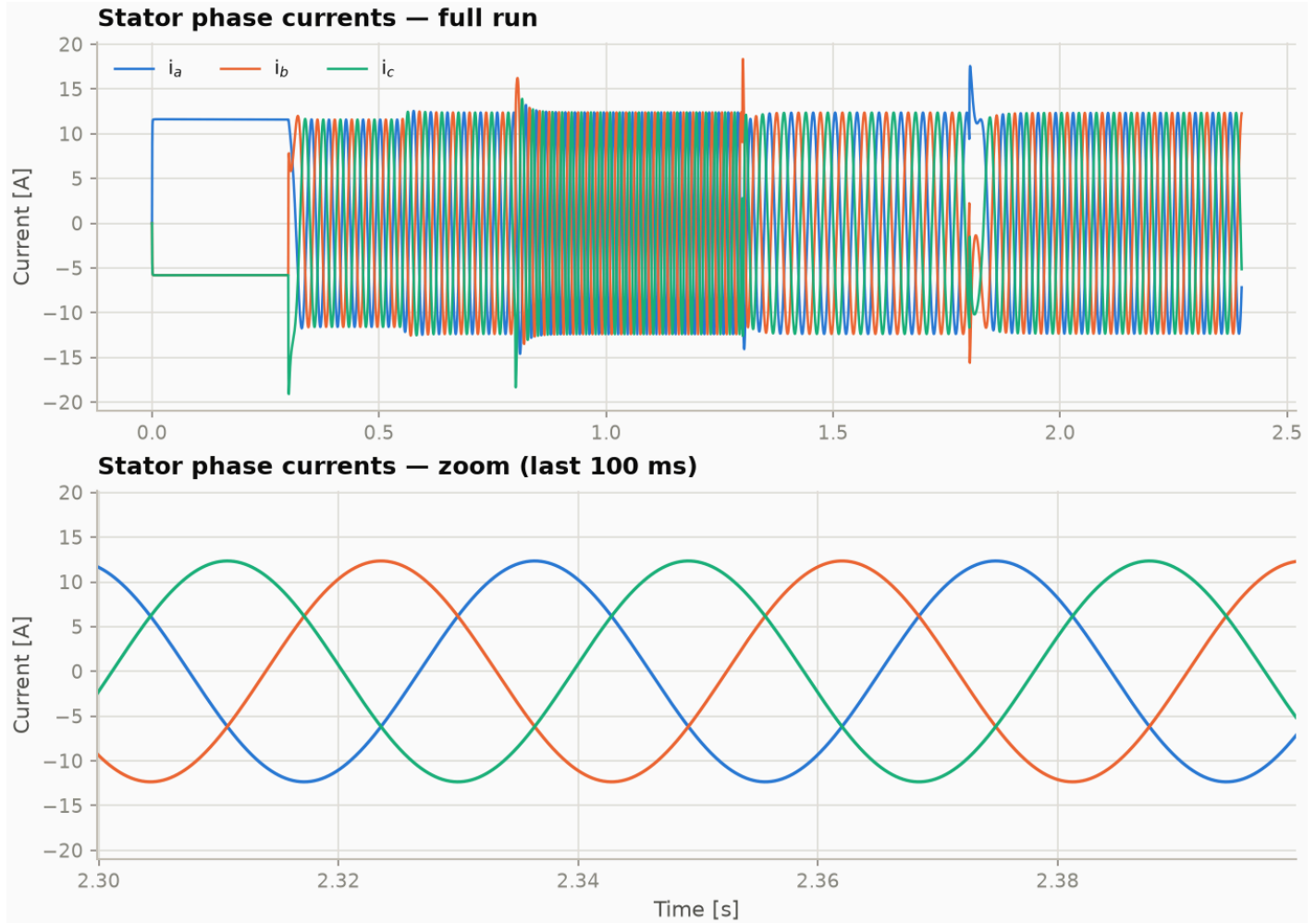
6.4 Currents in the rotor-flux frame



This figure is the decoupling proof. The flux-producing current i_d jumps to its reference 11.6 A at $t = 0$ and stays there *regardless of everything the q-axis does* — four speed steps, torque saturation, load engagement,

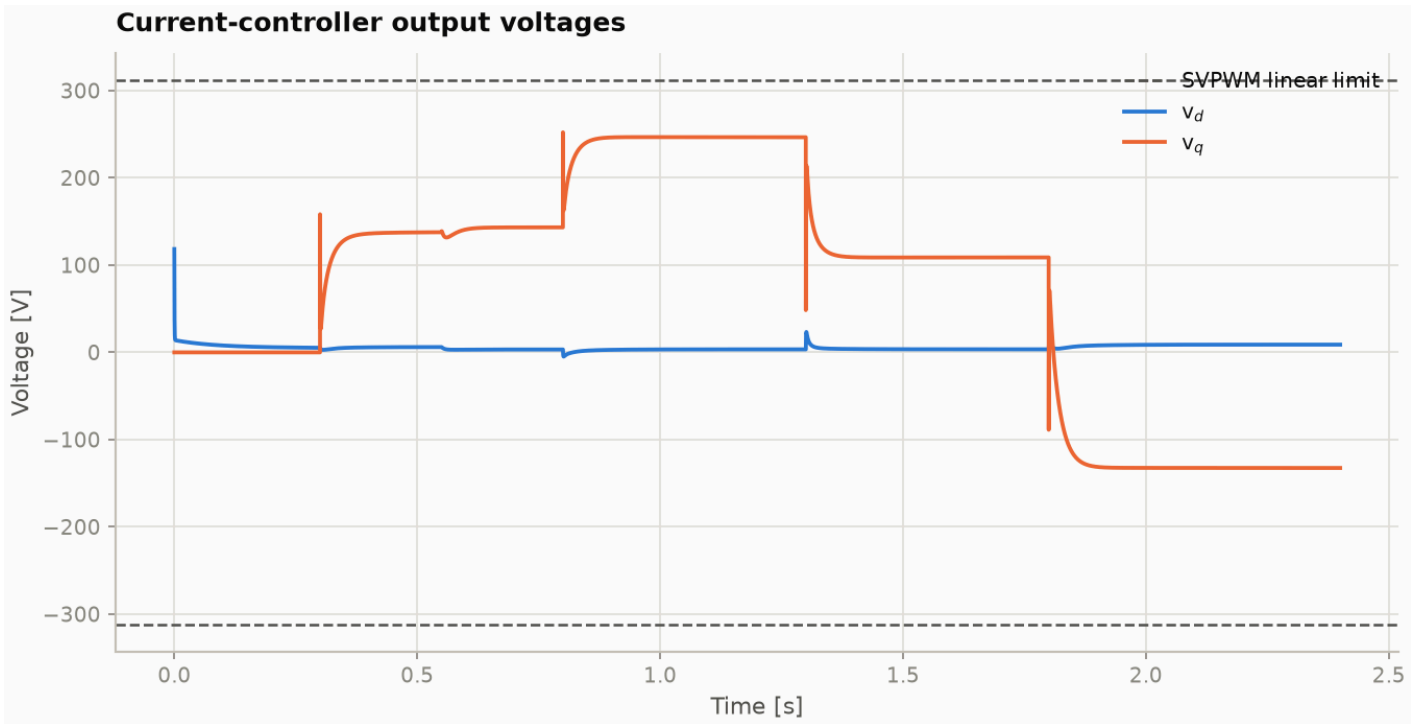
reversal. The torque-producing current i_q is an exact scaled copy of the torque trace (constant flux $\Rightarrow T_e \propto i_q$): +15 A spikes on the upward steps, -15 A spikes on the braking transitions (both are the 35 N·m limit divided by $K_t\psi_r$), and ≈ 4.3 A on the loaded plateaus. Reference and measured traces are indistinguishable at this scale — the 400 Hz current loops track within the line width, which is why the dashed reference curves are hidden behind the solid ones.

6.5 Stator phase currents



The phase-domain view ties the vector picture back to physical windings. During magnetization the currents are DC ($i_a = 11.6$ A, $i_b = i_c = -5.8$ A): the current vector stands still because the flux frame stands still. Each plateau then shows a distinct constant frequency proportional to its speed level ($f_e = (p\omega_m + \omega_{sl})/2\pi$), with short amplitude swells during the torque-limited step transitions and an amplitude increase when the load engages. During the reversal the frequency sweeps through a very low value as the electrical speed crosses zero. The zoom shows the final steady state at -800 rpm: three clean, balanced sinusoids of ≈ 12.3 A amplitude at ≈ 27 Hz (phase sequence reversed with respect to forward rotation). The peak current over the whole run is 19.1 A, inside the 25 A limit.

6.6 Controller output voltages



The q-axis voltage carries the back-EMF, so it steps between the levels the speed levels dictate (≈ 250 V at 1400 rpm, ≈ 110 V at 600 rpm, negative at -800 rpm), with short excursions toward the limit during the step transients; the d-axis voltage stays small ($\approx R_s i_d$ plus coupling corrections). The voltage vector remains inside the SVPWM linear limit of 311.8 V at all times — the operating envelope was chosen so that the drive never needs field weakening, which keeps the constant-flux assumption of Section 4.1 valid throughout.

6.7 Summary of quantitative results

Metric	Value
Step settling times, ± 10 rpm band (800 / 1400 / 600 / -800 rpm)	79 / 75 / 71 / 95 ms
Overshoot on any step	none
RMS speed error on settled plateaus	0.05 rpm
Maximum speed error on settled plateaus	0.25 rpm
Load-step dip (10 N·m at 800 rpm) / recovery	69 rpm / ≈ 70 ms
Final speed error (-800 rpm plateau)	0.0 rpm
Rotor flux at end (reference 0.80 Wb)	0.800 Wb
Orientation residual $\ \psi_{qr}\ $ at end	0.013 Wb (1.6 %)
Peak phase current (limit 25 A)	19.1 A

7. Control-oriented study: ramp and sinusoidal tracking

The step scenario of Section 6 exercises the drive as a *machine*; this second, separate scenario exercises the speed loop as a *servo*. It is run unloaded and with references gentle enough that no limit is ever reached, so the measured errors reflect the PI loop itself — not the physics of saturation or disturbances:

Interval	Reference	Purpose
0 – 0.30 s	0 rpm (flux build-up)	magnetize
0.30 – 0.90 s	ramp 0 → 1000 rpm	ramp tracking (type-2 behavior)
0.90 – 1.20 s	hold 1000 rpm	settle
1.20 – 1.70 s	sine 1000 ± 150 rpm, 2 Hz	slow sinusoidal tracking
1.70 – 2.30 s	sine 1000 ± 150 rpm, 5 Hz	faster sinusoidal tracking

7.1 What a PI loop can and cannot track — internal model principle

Zero steady-state tracking error requires the open loop to *contain a model of the reference signal* (the internal model principle). The speed loop's open-loop transfer function is approximately

$$L(s) = \underbrace{\left(K_p + \frac{K_i}{s} \right)}_{\text{PI}} \cdot \underbrace{\frac{1}{Js}}_{\text{mechanics}}$$

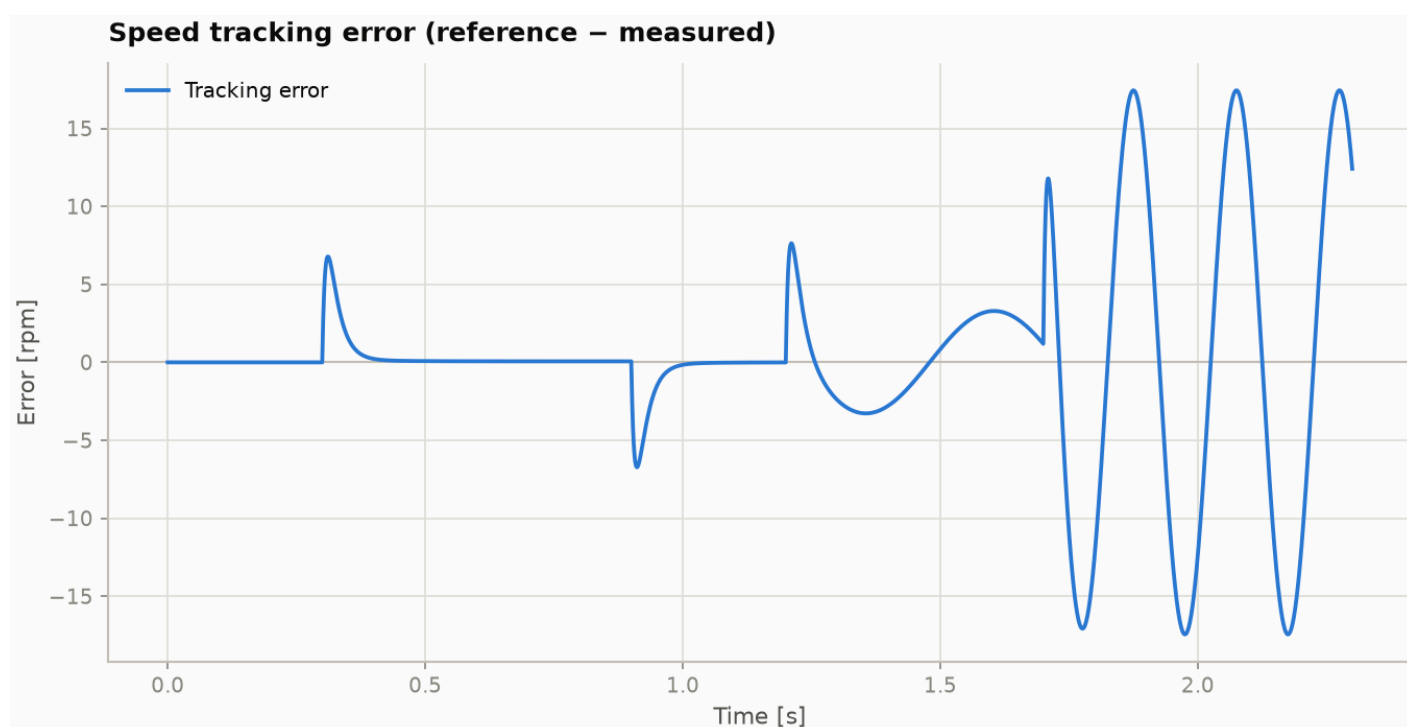
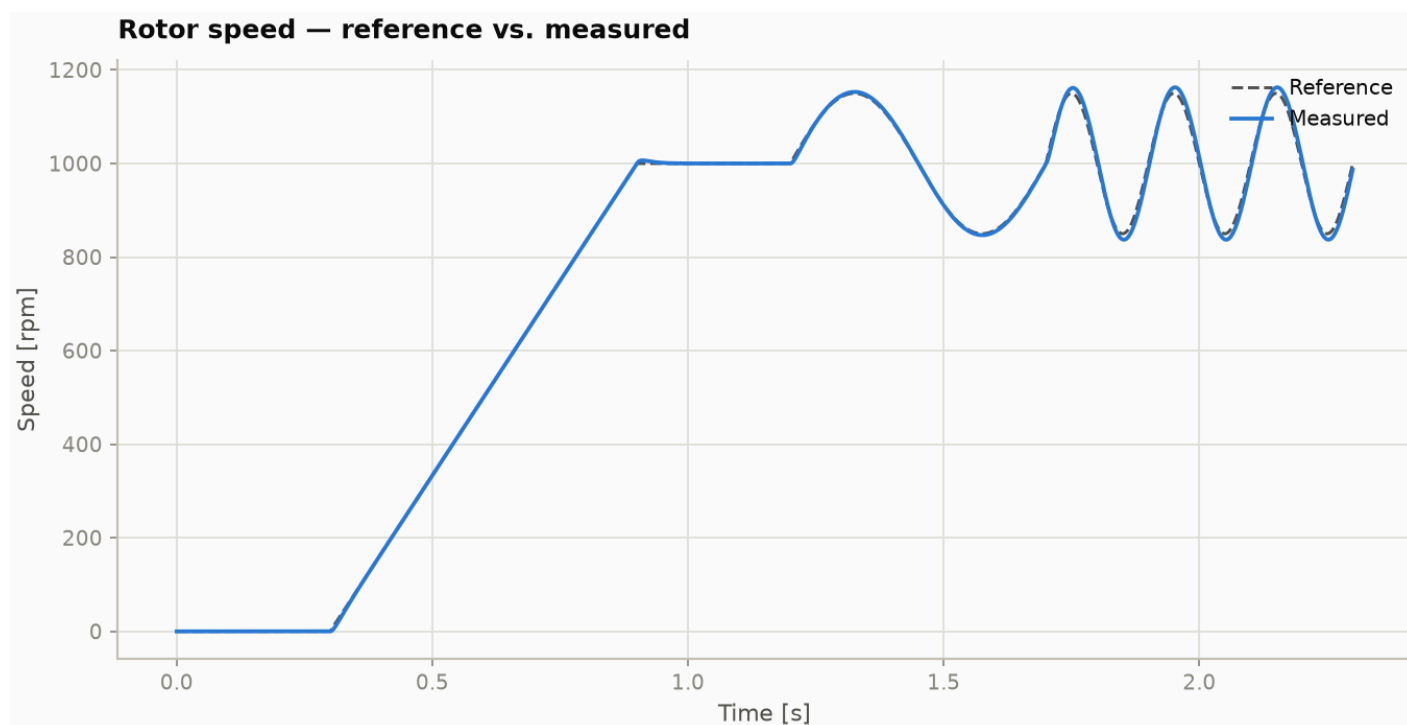
which contains **two** integrators — one from the PI, one from the inertia. The consequences, by reference class:

- **Steps** (one integrator needed): tracked with zero steady-state error — demonstrated throughout Section 6.
- **Ramps** (two integrators needed): the loop is *type 2*, so ramps are also tracked with zero steady-state error.
- **Sinusoids** (a resonant pole pair at the excitation frequency needed): the PI contains no such term, so a sinusoid is tracked with a **finite, frequency-dependent error**. From $E(s) = R(s)/(1 + L(s))$, a sine of amplitude A and angular frequency ω leaves the steady error amplitude

$$|E| \approx A \frac{J\omega^2}{|K_p j\omega + K_i|}$$

growing roughly with ω^2 at low frequency — the loop "runs out of gain" as the reference accelerates harder. Exact sinusoidal tracking would require adding that missing internal model (a proportional-resonant term) or a reference feed-forward path.

7.2 Results

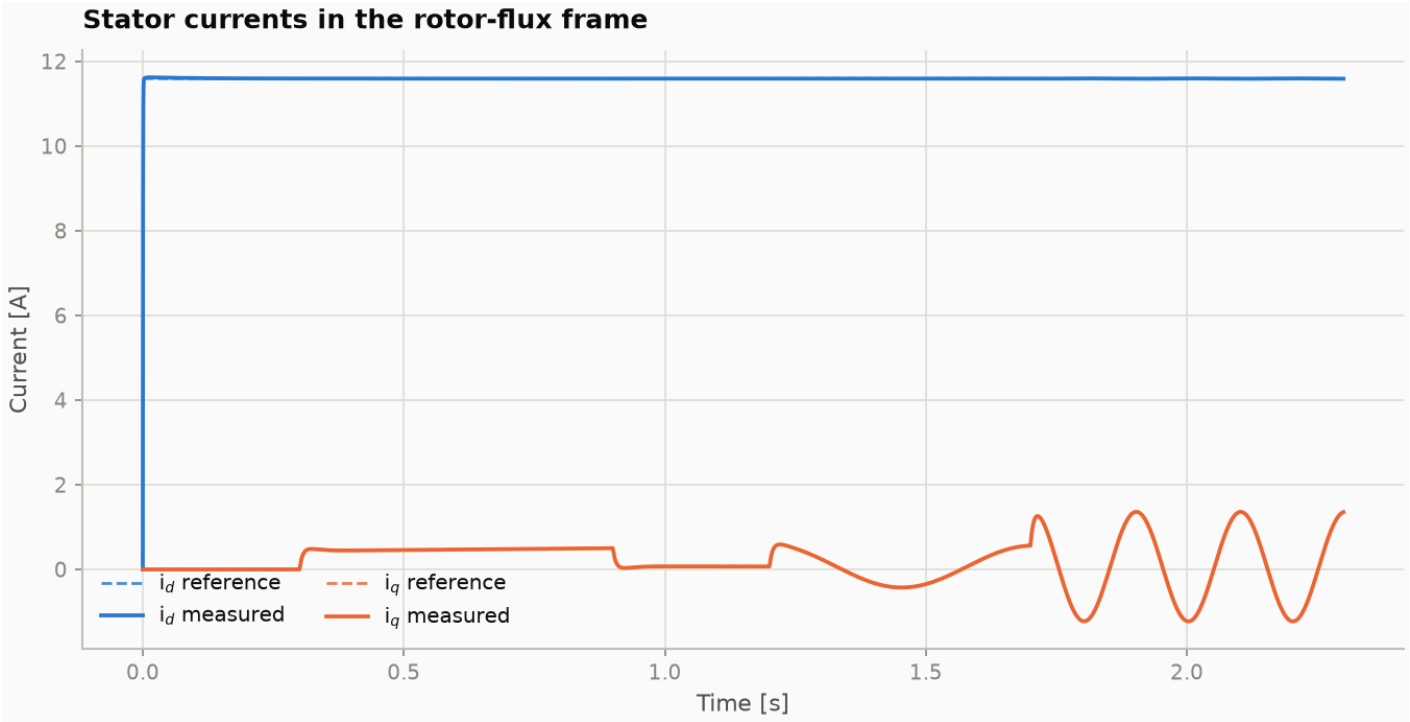


The speed trace and, more sharply, the tracking-error trace confirm each prediction quantitatively:

Segment	Measured error (max / RMS)	Predicted
Ramp, steady part	0.08 / 0.07 rpm	≈ 0 (type-2 loop)
Hold 1000 rpm	0.16 / 0.04 rpm	0
Sine 2 Hz $\pm$ 150 rpm	3.29 / 2.41 rpm	3.16 rpm (2.1 % of amplitude)
Sine 5 Hz $\pm$ 150 rpm	17.45 / 12.63 rpm	16.00 rpm (10.7 % of amplitude)

The ramp is followed with essentially zero error once the short transient at the slope discontinuity decays (the small ± 7 rpm blips at 0.30 s and 0.90 s are the loop absorbing the sudden change of reference *slope* — a ramp onset is a step in acceleration demand). The 2 Hz sine leaves a barely visible 3.3 rpm error; at 5 Hz the error

grows to 17.5 rpm — a factor ≈ 5.3 for a frequency factor 2.5, close to the ω^2 law (the residual difference from the prediction comes from the neglected friction and current-loop lag, both of which the formula ignores). The error is almost pure phase lag: its envelope is sinusoidal at the excitation frequency, in quadrature with the reference.



The dq-current view closes the loop on the FOC story: throughout the whole servo exercise i_d never moves from 11.6 A, while i_q supplies the small oscillating acceleration torque ($J \dot{\omega}$ never exceeds a few N·m here) — flux/torque decoupling holds during dynamic tracking exactly as it did in the step scenario.

The practical reading for drive design: a PI speed loop is the right — and sufficient — tool when the application's references are setpoints and load disturbances (the overwhelming industrial case). When a drive must *follow a trajectory* (positioning axes, electronic cams), the controller is augmented with feed-forward or resonant terms rather than by raising the PI gains, which would trade tracking for noise sensitivity and robustness.

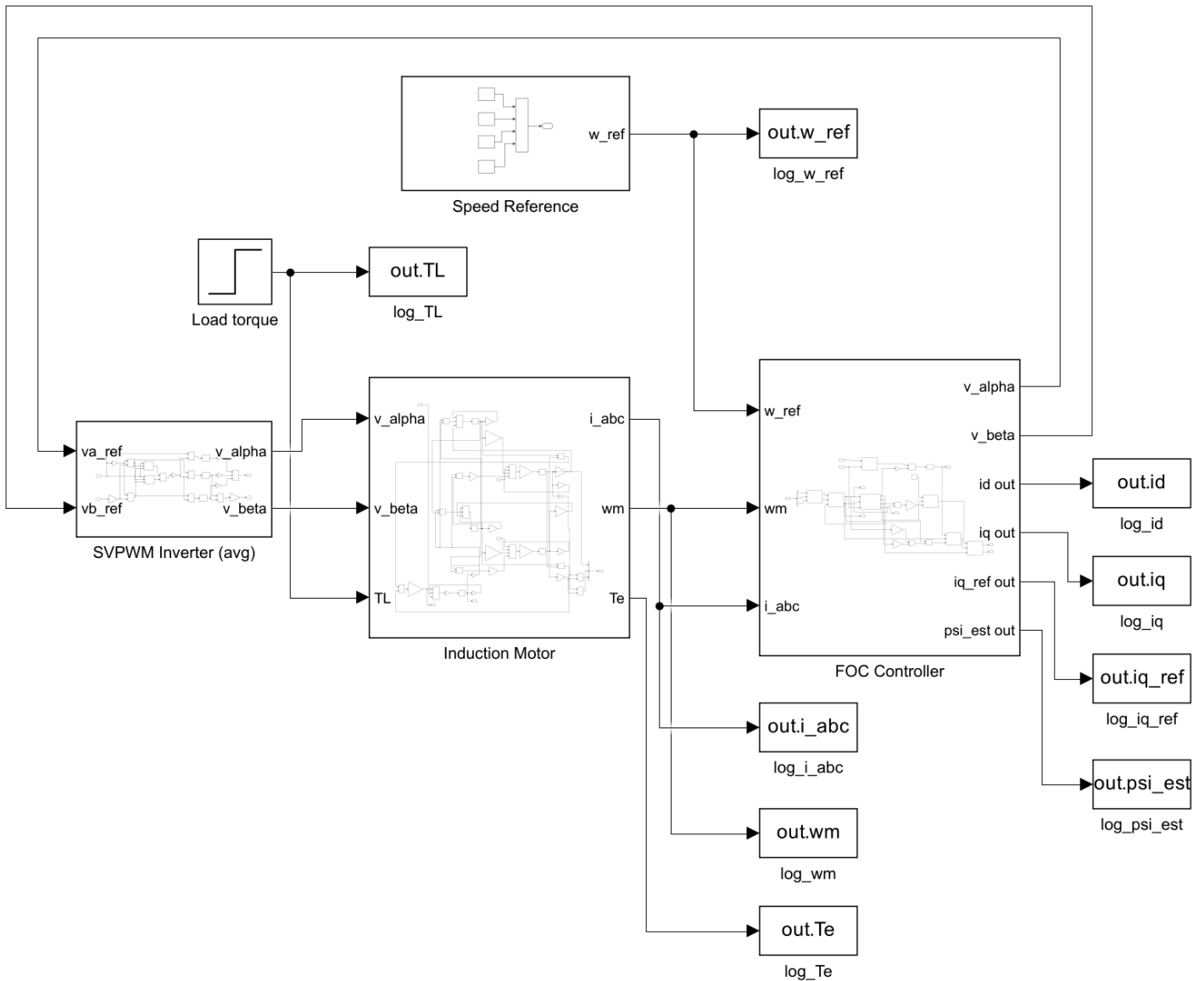
8. MATLAB / Simulink implementation

The same drive was rebuilt twice in Simulink (R2024b) to cross-check the Python results on an independent platform and to study the effect of a real switching inverter. Both versions use the machine data of Section 2.1, the gains of Section 4.5 and the speed-step scenario of Section 5; the ramp/sine tracking study of Section 7 is run only by the Python simulation.

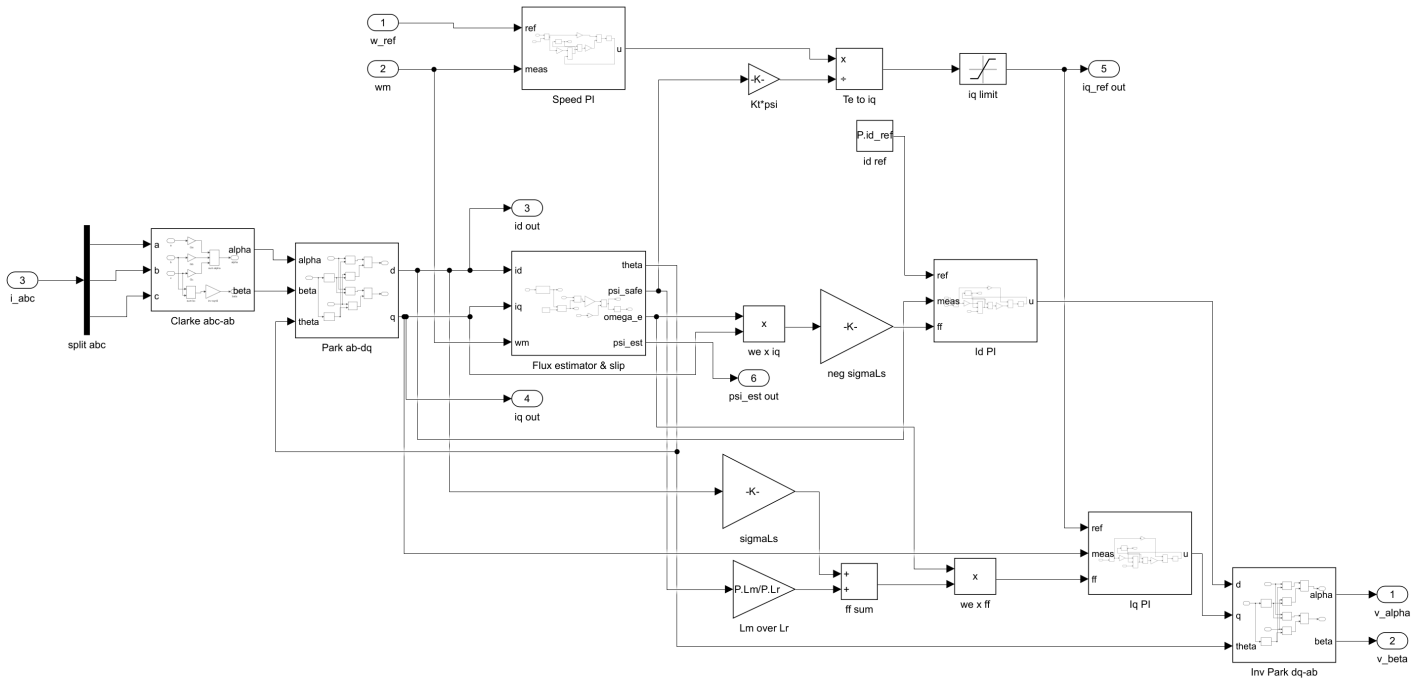
	Version 1 — <code>simulink_blocks/</code>	Version 2 — <code>simulink_library/</code>
Built from	fundamental Simulink blocks only	Simscape Electrical (SPS) components
Machine	hand-built dq equations	Asynchronous Machine SI Units
Inverter	average-value SVPWM	Universal Bridge (IGBT) + PWM Generator
Solver	<code>ode23tb</code> , variable step	fixed step, discrete, 2.5 μ s
Run time (2.4 s)	seconds	$\sim$ 2–3 min

Both models are parameterised entirely from the struct `P` built by `foc_init.m`: every block dialog references `P` rather than a hard-coded number, so the two Simulink versions and the Python simulation always run on the same machine data and the same gains.

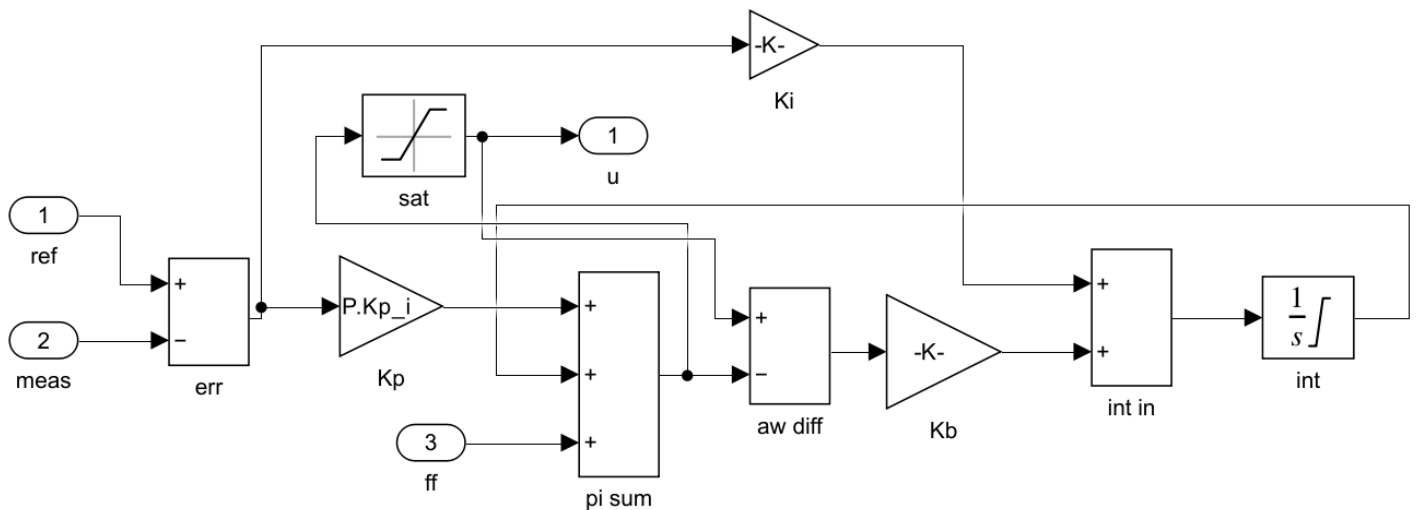
8.1 Version 1 — built from fundamental blocks



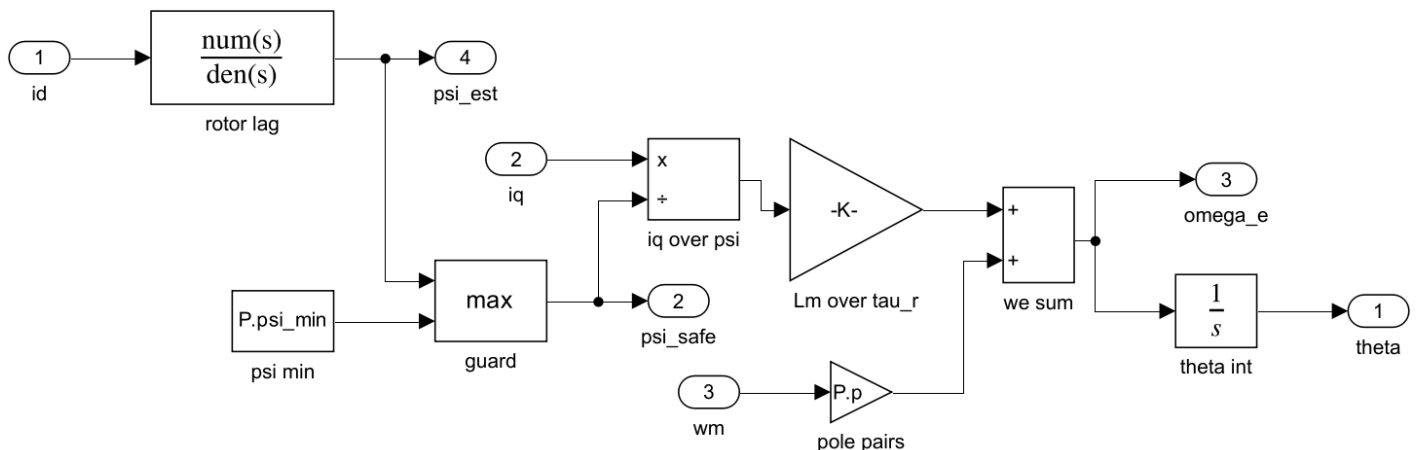
The top level is the classic closed loop: the speed reference feeds the FOC controller, which drives the averaged SVPWM inverter, which feeds the motor model; the measured phase currents and rotor speed return to the controller.



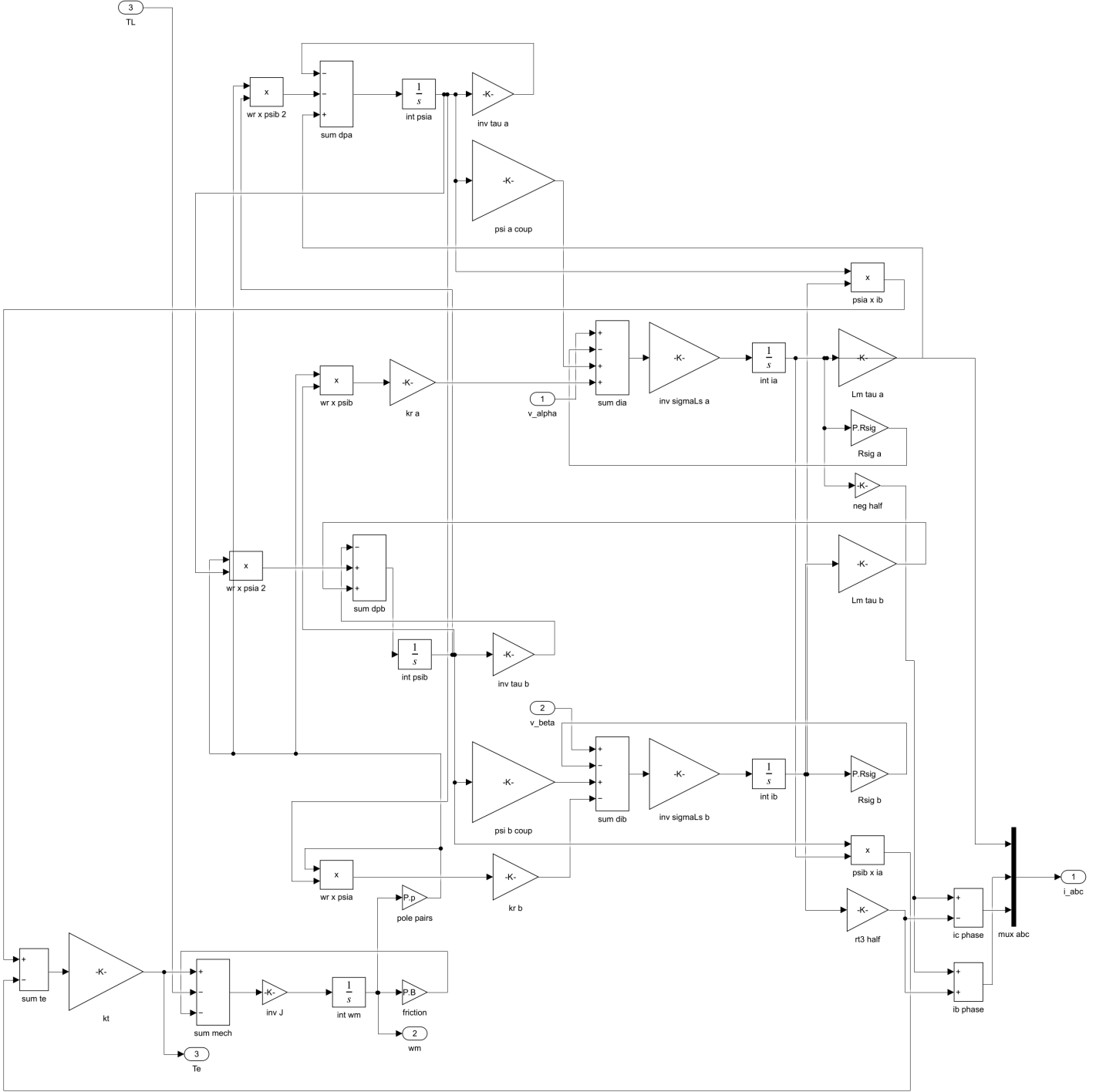
Inside the controller are the blocks of Section 4.3 — Clarke, Park, the flux estimator and slip integrator, the speed PI, the two current PIs with their decoupling feed-forward, and the inverse Park.



Each PI is built from a gain, an integrator and a saturation, plus the **back-calculation anti-windup loop**: the difference between the saturated and unsaturated output is fed back into the integrator through **Kb**. This is not cosmetic — with plain integrator clamping the integral state charges to the full torque limit during every saturated step, and the drive overshoots to about 960 rpm on the 800 rpm step. With back-calculation the overshoot disappears entirely and the settling times improve by roughly 20 %.

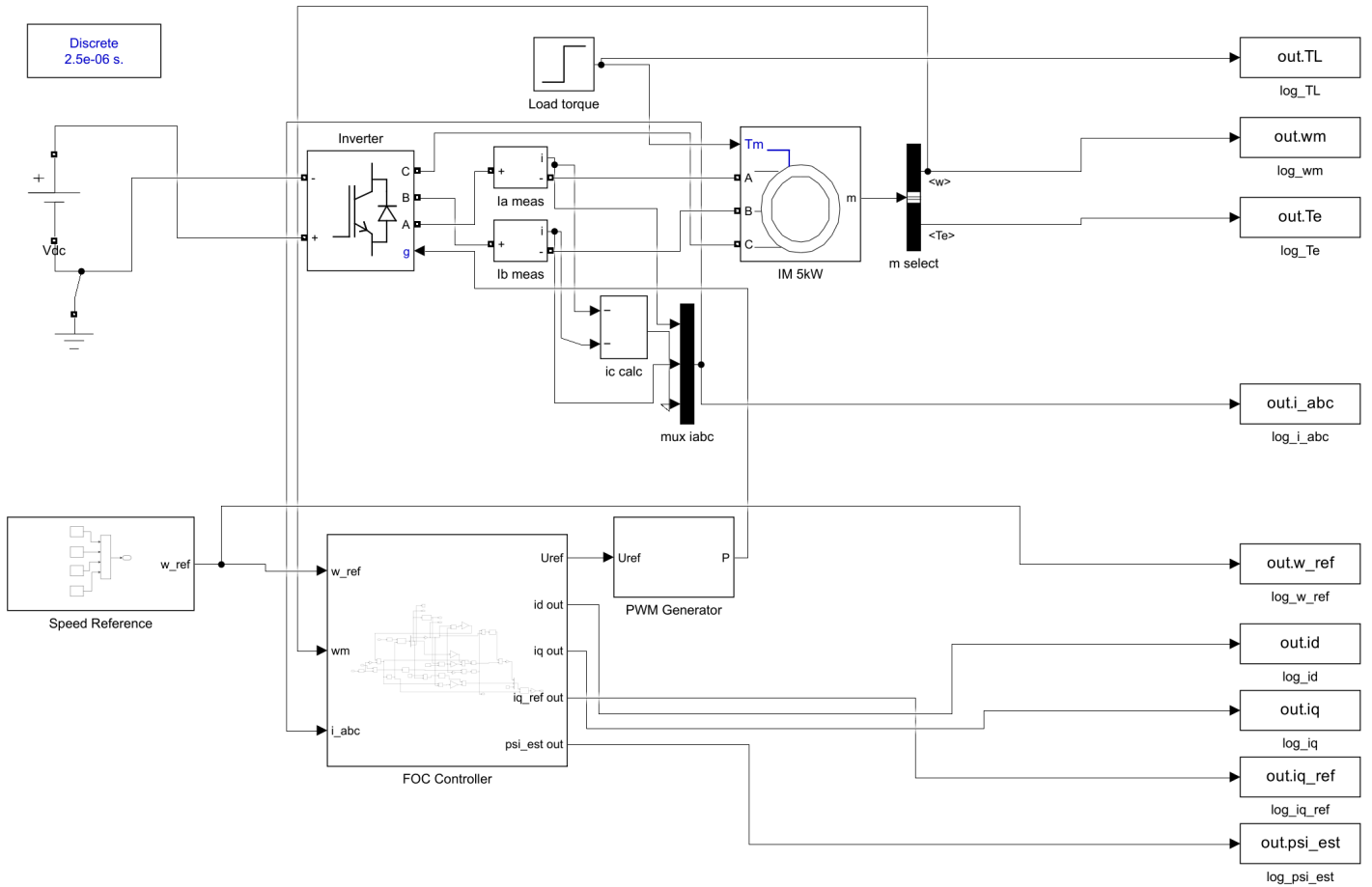


The estimator implements $\hat{\psi}_r = \frac{L_m}{1+s\tau_r} i_d$, guards it against division by zero, forms the slip $\omega_{sl} = L_m i_q / (\tau_r \psi_r)$ and integrates $\omega_e = p \omega_m + \omega_{sl}$ into the flux angle.

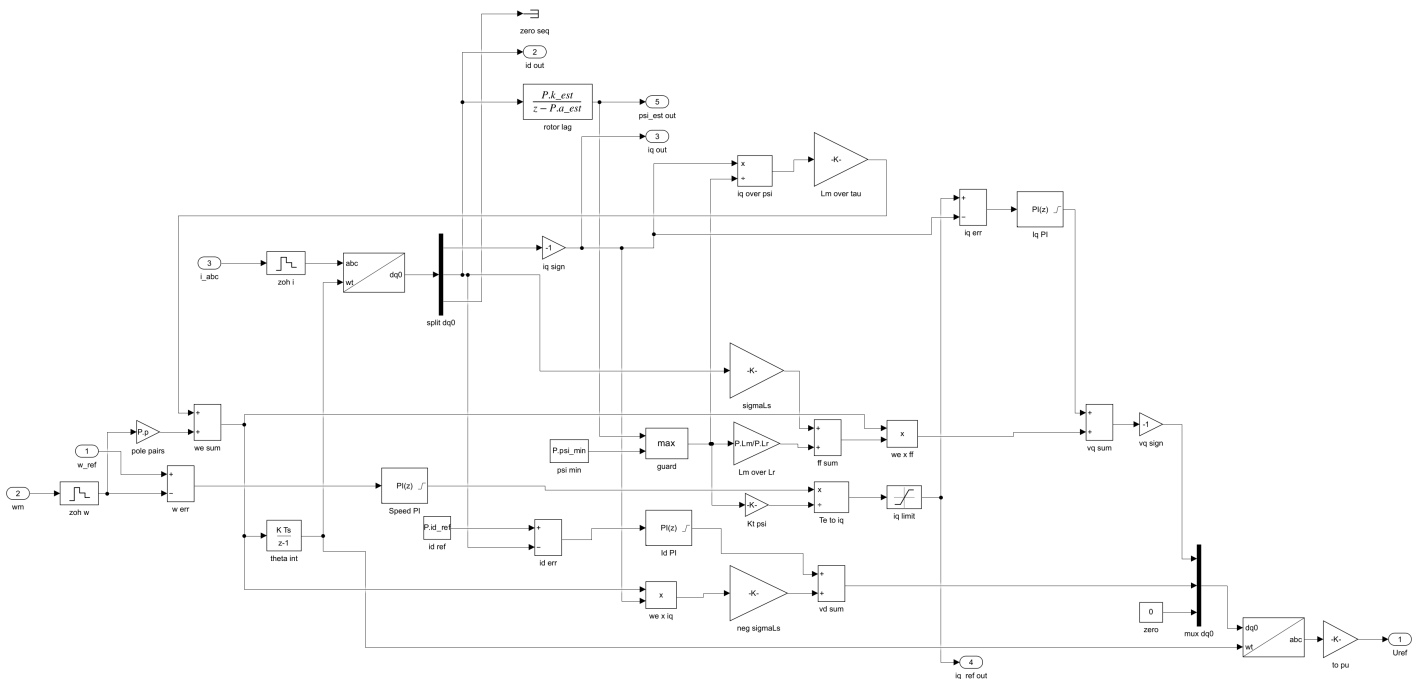


The machine is the state-space model of Section 2.2 drawn out explicitly: four integrators for $i_{s\alpha}$, $i_{s\beta}$, $\psi_{r\alpha}$, $\psi_{r\beta}$, the cross-coupling products with ω_r , the torque equation and the mechanical integrator.

8.2 Version 2 — built from library components



Here the power stage is physical: a DC voltage source, an IGBT Universal Bridge switching at 10 kHz, current measurement blocks and the Asynchronous Machine, all solved by `powergui` in discrete mode. The controller subsystem keeps the same structure but uses the library `abc to dq0` / `dq0 to abc` transforms and discrete PID blocks.

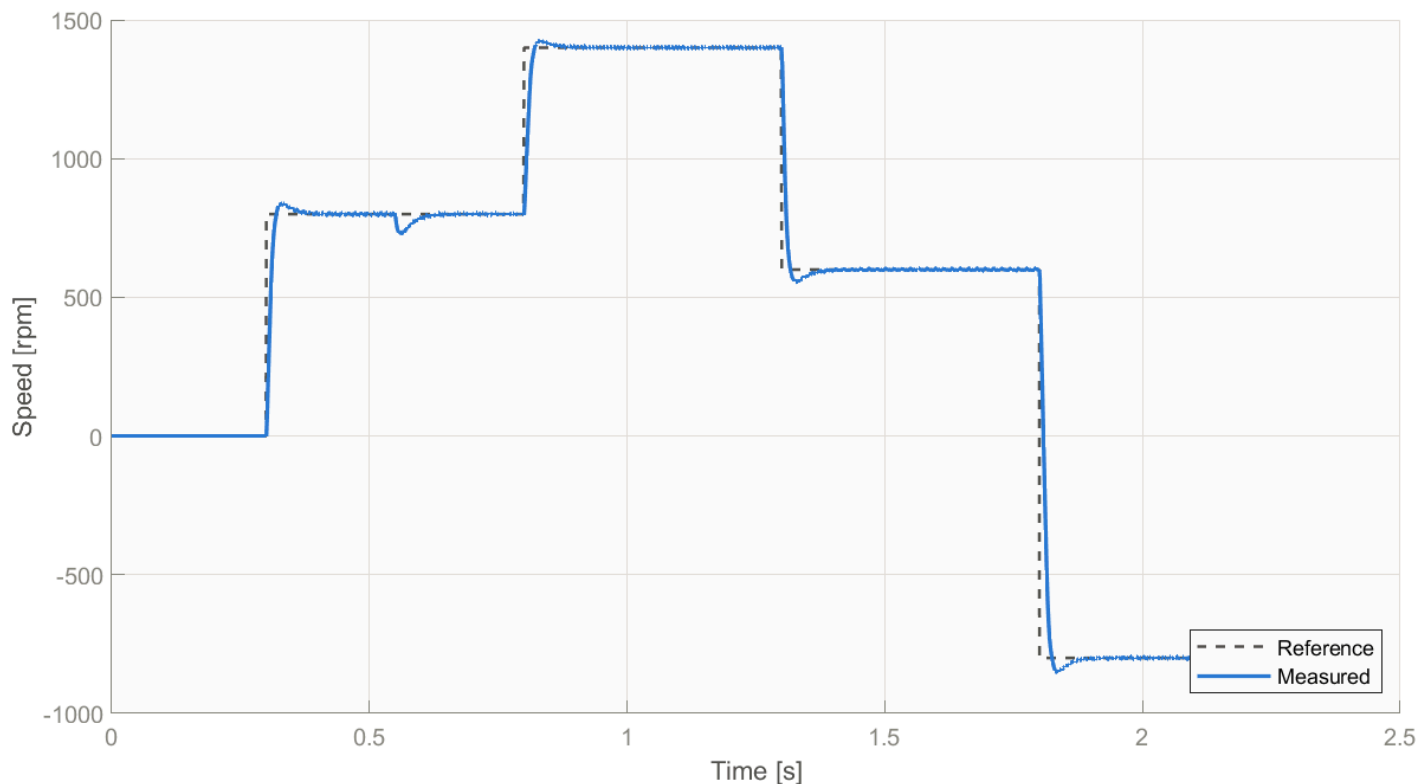


One implementation detail is worth recording because it is invisible until the drive fails to magnetise: the SPS transform blocks are **sine-referenced** ($d_{sps} = \frac{2}{3} \sum x_k \sin(\omega t - k\frac{2\pi}{3})$) whereas the Park transform of Section 3.3 is cosine-referenced. The two are related by $q_{sps} = d$ and $d_{sps} = -q$, so the controller contains an

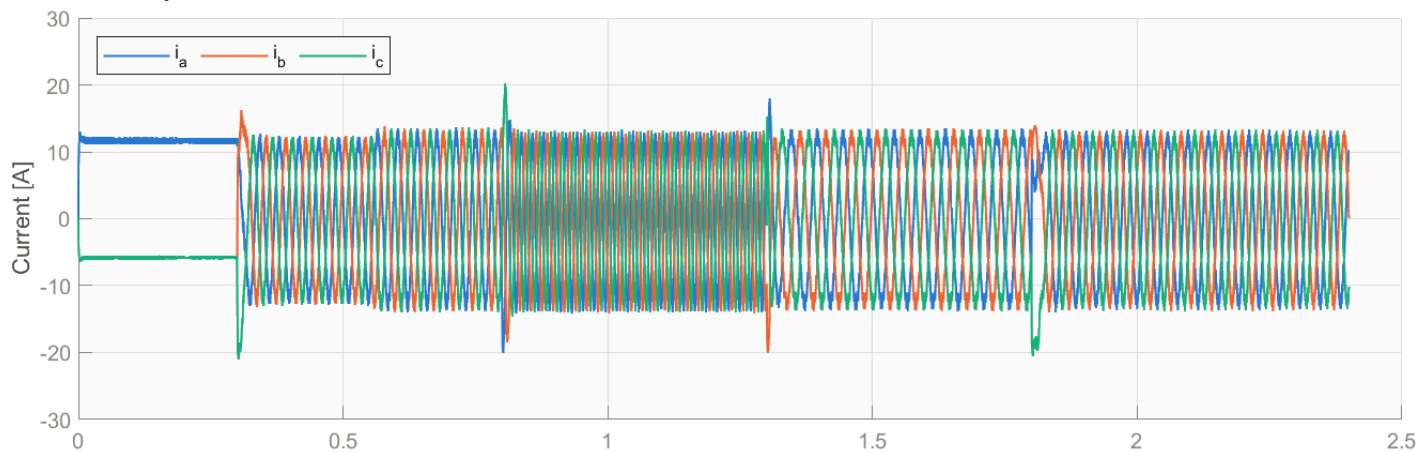
explicit axis-swap and sign inversion on both the measurement and the actuation path. Without it the reference frame sits 90° away from the rotor flux and field orientation is lost completely.

8.3 Results and cross-platform comparison

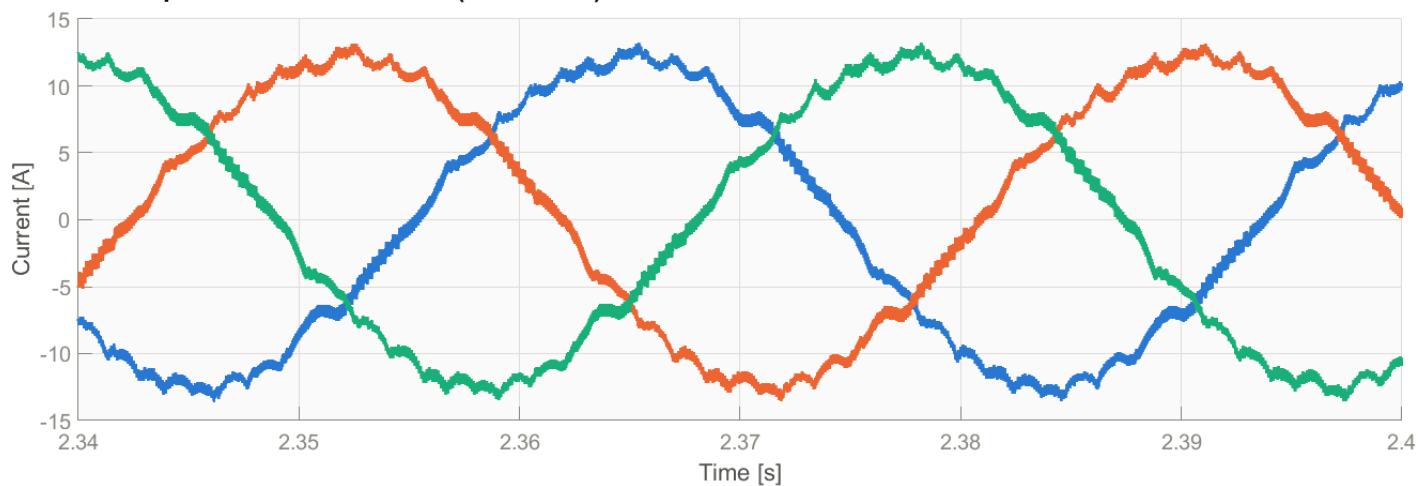
Rotor speed - reference vs. measured



Stator phase currents - full run



Stator phase currents - zoom (last 60 ms)



The switching model reproduces the same staircase response as Sections 6.2 and 6.5, with one visible difference: the phase currents now carry the PWM ripple that the averaged models smooth away, and the speed trace shows a corresponding fine ripple on each plateau.

Metric	Python	Simulink V1	Simulink V2
Settling times [ms]	79 / 75 / 71 / 95	70 / 68 / 52 / 59	63 / 62 / 67 / 72
Plateau error, max [rpm]	0.25	0.16	4.94
Load-step dip [rpm]	69.0	69.2	69.6
Rotor flux at end [Wb]	0.800	0.800	0.800
Peak phase current [A]	19.1	19.9	20.6

The three implementations agree on every physically meaningful quantity. The load-step dip — which depends only on the machine inertia and the speed-loop bandwidth, not on the modelling detail — matches to within 1 %, and the rotor flux lands on its reference in all three. The differences that remain are explained by what each model represents: Python runs a strictly discrete 100 μ s controller (slightly slower settling), Version 1 a continuous controller with an averaged inverter (fastest), and Version 2 a true switching bridge whose ripple accounts for the larger plateau error and the slightly higher peak current. That three independent implementations converge on the same numbers is the strongest available evidence that the model, the control design and the parameter set are all correct.

9. Conclusion

A complete field-oriented drive was modeled and validated in simulation for a 5 kW induction machine. The two-stage coordinate transformation — Clarke to collapse the redundant three-phase system onto two orthogonal axes, then Park to freeze the rotation at the rotor-flux angle — converts the coupled AC machine into two DC-like channels, and the indirect FOC structure (slip-based angle generation, current-model flux estimator, cascaded PI loops with decoupling and anti-windup) exploits that structure fully. The simulated drive holds rotor flux constant to within measurement thickness while the torque channel independently executes a staircase of speed steps — including regenerative braking and a full four-quadrant reversal — settling each step in under 100 ms with zero steady-state error; disturbance rejection and all actuator limits behave as designed. The dedicated tracking study of Section 7 completes the control picture: the loop is type 2 (zero error on steps *and* ramps), while sinusoidal references reveal the expected finite, ω^2 -growing error that only an added internal model or feed-forward could remove — measured and predicted values agreeing within ≈ 9 %. The two Simulink implementations of Section 8 confirm the design independently: the fundamental-block model and the switching Simscape Electrical model land on the same settling times, the same 69 rpm load dip and the same rotor flux as the Python simulation, with the switching model additionally showing the PWM ripple that the averaged models omit.

Limitations and outlook: the study assumes ideal current/speed sensors and constant parameters, whereas the indirect scheme's slip gain depends on the rotor time constant τ_r , which drifts with temperature — an online τ_r adaptation or a closed-loop (direct) flux observer would harden the design. Field weakening would extend operation above base speed, which is the natural next step now that the switching behaviour has been validated on the Simscape Electrical model. Beyond that, a sensorless variant replacing the measured speed with an observer would remove the encoder, and moving the tuning from fixed bandwidths to a robustness-based design would address the parameter sensitivity directly.

References

1. F. Blaschke, "The principle of field orientation as applied to the new transvektor closed-loop control system for rotating-field machines," *Siemens Review*, vol. 34, pp. 217–220, 1972.
2. K. Hasse, *Zur Dynamik drehzahl geregelter Antriebe mit stromrichter- gespeisten Asynchron-Kurzschlussläufermaschinen*, PhD dissertation, TH Darmstadt, 1969.
3. W. Leonhard, *Control of Electrical Drives*, 3rd ed., Springer, 2001.
4. P. C. Krause, O. Wasynczuk, S. D. Sudhoff, *Analysis of Electric Machinery and Drive Systems*, 3rd ed., Wiley-IEEE Press, 2013.
5. P. Vas, *Vector Control of AC Machines*, Oxford University Press, 1990.
6. B. K. Bose, *Modern Power Electronics and AC Drives*, Prentice Hall, 2002.